Open-Source Implementations of the RISC-V Vector Extension: A Comparative Overview of Ara, Spatz, Vicuna, RS5, and Marian

Technical Survey

November 3, 2026

Abstract

The RISC-V Vector Extension (RVV), ratified in its version 1.0 form, provides a scalable, length-agnostic SIMD paradigm that is particularly attractive for embedded, edge, and high-performance computing. This article presents a detailed scientific overview of the major mature open-source hardware implementations that support full or substantial subsets of RVV, with emphasis on element widths of 16, 32, and 64 bits. We examine six prominent designs—Ara (and its evolution Ara2), Spatz, Vicuna, Vicuna 2, RS5, and Marian—comparing their microarchitectures, supported extension subsets (`Zve32x`, `Zve32f`, `Zve64d`, full `V`, and `Zvk`), integration models, configurability, and reported Power-Performance-Area (PPA) characteristics. A multi-criteria ranking is derived that balances ISA completeness, silicon maturity, energy efficiency, area efficiency, and ecosystem readiness. The survey concludes with observations on remaining gaps and promising directions for future open-source vector processors.

Contents

1 Introduction **2**

2 Background on the RISC-V Vector Extension **2**
2.1 Key Architectural Concepts 2
2.2 Implementation Challenges 2

3 Detailed Examination of the Implementations **3**
3.1 Ara and Ara2 3
3.2 Spatz 3
3.3 Vicuna and Vicuna 2 4
3.4 RS5 4
3.5 Marian 4

4 Comparative Analysis and Ranking **4**
4.1 Interpretation of the Ranking 5

5 Discussion and Future Directions **5**

6 Conclusion **6**

1 Introduction

The RISC-V Vector Extension (RVV) [1] introduces a flexible vector architecture whose vector length (`VLEN`) is implementation-defined and whose element width (`SEW`/`EEW`) can be selected dynamically. The specification defines both a full application-class extension `V` (requiring at least `Zve64d` and a minimum `VLEN` of 128 bits) and a family of embedded subsets (`Zve32x`, `Zve32f`, `Zve64x`, `Zve64f`, `Zve64d`). These subsets allow implementers to trade ISA completeness for silicon cost, making RVV attractive across the spectrum from tiny microcontrollers to high-performance accelerators.

Open-source hardware has played a decisive role in the rapid adoption of RVV. Several academic and industrial groups have released production-quality SystemVerilog (or Chisel) designs that can be synthesized for FPGA or ASIC targets. This article focuses on six of the most mature and widely referenced implementations:

- **Ara /** Ara2 (PULP Platform, ETH Zürich & University of Bologna) [2, 3];
- **Spatz** (PULP Platform) [4];
- **Vicuna** and its successor **Vicuna 2** [5, 6];
- **RS5** (GAPH, PUCRS) [7];
- **Marian** (SoC Hub / Tampere University) [8].

We deliberately exclude pure software simulators, incomplete academic prototypes that have not been released under a permissive license, and commercial closed-source IP. The goal is a technically rigorous comparison that can guide architects choosing an open-source vector engine for a new SoC or research platform.

2 Background on the RISC-V Vector Extension

2.1 Key Architectural Concepts

RVV organizes computation around 32 vector registers (`v0`–`v31`), each of width `VLEN` bits. The active vector length `VL` and the element width `SEW` are controlled by the `vsetvli`/`vsetivli` family of instructions. Grouping of registers (`LMUL`) allows longer logical vectors. Memory operations support unit-strided, strided, and indexed (gather/scatter) addressing; arithmetic units provide integer, fixed-point, and floating-point operations with widening and narrowing variants; mask and permutation instructions complete the ISA.

The embedded subsets are defined as follows (simplified):

Extension	Min. `VLEN`	Supported EEW	FP support
`Zve32x`	32	8, 16, 32	none
`Zve32f`	32	8, 16, 32	FP32
`Zve64x`	64	8, 16, 32, 64	none
`Zve64f`	64	8, 16, 32, 64	FP32
`Zve64d`	64	8, 16, 32, 64	FP32+FP64
`V`	128	…	full (`Zve64d`+)

2.2 Implementation Challenges

Designing an efficient RVV engine involves several well-known tensions:

- **Lane-based vs. centralized microarchitecture.** Scaling the number of functional units while keeping the Vector Register File (VRF) multi-ported and low-latency.

- **Chaining and hazard detection.** Allowing a dependent instruction to start as soon as the first elements of its sources are ready.
- **Memory bandwidth.** Matching the compute throughput of many lanes to a realistic memory hierarchy.
- **Scalar–vector interface.** Efficient offload of vector instructions from a scalar host core (CVA6, Snitch, Ibex/CV32E40X, etc.).
- **Configurability vs. verification cost.** Highly parametric designs increase the verification burden.

3 Detailed Examination of the Implementations

3.1 Ara and Ara2

Ara is a lane-based vector co-processor that attaches to the CVA6 (formerly Ariane) application-class core. Each lane contains a slice of the VRF, an integer ALU, a multiplier, and a floating-point unit. A central sequencer, a Vector Load/Store Unit (VLSU), and a Slide Unit (SLDU) coordinate the lanes. The design is parametric in the number of lanes (typically 2–16).

Ara originally implemented an early draft of RVV; Ara2 brought the design to full compliance with the frozen RVV 1.0 specification, including all integer, floating-point, mask, and permutation instructions required by the `V` extension (ELEN = 64). Silicon results in GlobalFoundries 22 nm FD-SOI demonstrate:

- Clock frequencies above 1 GHz (typical corner);
- Peak energy efficiency of approximately 38–41 DP-GFLOPS/W;
- Functional-unit utilization exceeding 95 % on long-vector matrix-multiplication kernels;
- Area that scales roughly linearly with the number of lanes, with the SLDU becoming a critical path for large configurations.

Ara remains the reference high-performance open-source RVV engine and the foundation upon which Marian is built.

3.2 Spatz

Spatz targets a different point in the design space: compact vector processing elements for tightly-coupled multi-core clusters that share a multi-banked L1 scratchpad. Each Spatz instance is paired with a lightweight Snitch scalar core and features a very small latch-based VRF (typically 2 KiB, corresponding to `VLEN` = 512). The architecture comprises a Vector Arithmetic Unit (VAU) with multiple transprecision FPUs and an integer unit, a VLSU with multiple narrow memory ports, and a Vector Slide Unit.

Spatz implements the `Zve64d` subset. In a dual-core cluster fabricated in GlobalFoundries 12 nm, the design achieves:

- $> 95\,\%$ FPU utilization on double-precision matrix multiplication and 2-D convolution;
- Energy efficiency approaching 100 DP-GFLOPS/W;
- Outstanding area efficiency (reported 171 DP-GFLOPS/W/mm^2).

The key insight of Spatz is that a modest VRF is sufficient when the compute-to-control ratio is high and the memory system is local and high-bandwidth.

3.3 Vicuna and Vicuna 2

Vicuna is an embedded vector co-processor that communicates with a scalar host (Ibex or CV32E40X) through the CORE-V eXtension Interface (CV-X-IF). It targets the `Zve32x` subset (8/16/32-bit integer and fixed-point). The microarchitecture is organized as a set of configurable pipelines that can host different functional units (VLSU, VALU, VMUL, VSLD, VELEM, and later VDIV).

Vicuna 2 extends the original design with:

- Complete support for vector division (finishing `Zve32x`);
- Optional `Zve32f` (single-precision floating-point) and `Zvfh` (half-precision) via integration of the CV-FPU;
- Greater flexibility in pipeline width (`VLANE_W`) and register width (`VREG_W`).

Vicuna and Vicuna 2 excel in area-constrained embedded systems where 64-bit elements and double-precision floating-point are unnecessary. Their high degree of configurability makes them attractive for design-space exploration studies.

3.4 RS5

RS5 is a 4-stage in-order RV32 core developed at PUCRS. It optionally implements a subset of `Zve32x` together with a configurable `VLEN` (and `LLEN`). Because the scalar core itself does not implement the `F` extension, floating-point vector support is absent. The vector unit is integrated more tightly than a pure co-processor, sharing the pipeline resources of the host core.

RS5 is smaller and less mature than Ara or Spatz, yet it demonstrates that a modest vector capability can be added to a classic 5-stage (or 4-stage) microcontroller-class pipeline with limited area overhead. It also supports a variety of other optional extensions (bit-manipulation, cryptography, custom Kyber instructions), making it a flexible platform for educational and research SoCs.

3.5 Marian

Marian is a derivative of Ara2 that adds the full RISC-V Vector Cryptography Extension suite (`Zvk`, including `Zvkned`, `Zvknh`, `Zvkg`, `Zvksed`, `Zvksh`, etc.). A dedicated crypto unit is inserted into the Ara pipeline; it receives operands from the VRF, performs AES, SHA-2, SM3/SM4 and GCM operations on 128- or 256-bit element groups, and writes results back. Marian has been prototyped on a Xilinx Virtex UltraScale+ FPGA and is targeted for a 22 nm tape-out.

Because Marian inherits Ara's mature vector infrastructure, it inherits full `V` support while adding high-throughput bulk cryptography. Measured speed-ups on AES and SHA kernels relative to pure software (OpenSSL) reach one to two orders of magnitude in cycle count and instruction count.

4 Comparative Analysis and Ranking

We evaluate the six designs along five orthogonal axes, each scored on a 1–5 scale (higher is better):

1. **ISA Completeness** — breadth of supported RVV subsets and additional extensions;
2. **Silicon / FPGA Maturity** — existence of silicon results, rigorous verification, and long-term maintenance;
3. **Energy Efficiency** — reported GFLOPS/W or equivalent metrics on representative kernels;
4. **Area Efficiency / Compactness** — suitability for area-constrained or multi-core cluster designs;
5. **Ecosystem & Configurability** — quality of documentation, toolchain support, and ease of parameterization.

Table 1: Multi-criteria ranking of the surveyed open-source RVV implementations (scores 1–5).

Design	ISA	Maturity	Energy	Area	Ecosystem	**Total**
Ara / Ara2	5	5	5	3	5	**23**
Spatz	4	4	5	5	4	**22**
Marian	5	3	4	3	3	**18**
Vicuna 2	3	3	3	4	4	**17**
Vicuna	2	3	3	4	3	**15**
RS5	2	2	2	4	3	**13**

4.1 Interpretation of the Ranking

- **Ara/Ara2** occupies the top position because it combines full RVV 1.0 compliance, extensive silicon characterization, excellent energy efficiency on long vectors, and a well-maintained open-source repository. Its only relative weakness is area: the design is optimized for high absolute throughput rather than minimal silicon footprint.
- **Spatz** is a very close second. By deliberately constraining the VRF size and targeting shared-L1 clusters it achieves superior area efficiency and still delivers state-of-the-art energy efficiency. It is the preferred choice when many compact vector engines must be integrated into a single die.
- **Marian** inherits Ara's strengths and adds a unique capability (vector cryptography). Its lower maturity score reflects the more recent public release and the still-ongoing ASIC flow.
- **Vicuna 2** is the strongest pure-embedded contender. Its support for `Zve32f`/`Zvfh` and the highly configurable pipeline organization make it ideal for design-space exploration of low-power 32-bit vector systems.
- **Vicuna** (original) remains useful but is superseded by Vicuna 2 for new designs.
- **RS5** demonstrates that a modest vector unit can be added to a classic microcontroller pipeline; it is attractive for educational platforms and highly constrained SoCs, yet lags in both ISA coverage and published PPA data.

5 Discussion and Future Directions

Several observations emerge from the comparison.

First, the open-source RVV landscape has matured rapidly: full 64-bit floating-point vector support (Ara, Spatz) and even vector cryptography (Marian) are now available under permissive licenses. Second, there is a clear bifurcation between high-throughput lane-based designs optimized for long vectors (Ara) and compact designs optimized for short-to-medium vectors inside multi-core clusters (Spatz). Third, the embedded 32-bit space (Vicuna 2, RS5) still offers substantial room for further optimization, particularly with respect to memory hierarchy and power gating.

Open challenges that remain largely unaddressed by the current generation of open-source engines include:

- Robust support for precise traps and virtual memory (required for full Linux capability with the `V` extension);
- Efficient handling of very short vectors without under-utilizing wide datapaths;
- Formal verification of the more complex units (especially the crypto datapaths of Marian);

- Standardized, high-bandwidth scalar–vector interfaces beyond CV-X-IF and the custom interfaces used by Ara/Spatz.

Future work is likely to focus on multi-core vector clusters that combine the best of Ara-style and Spatz-style microarchitectures, on matrix extensions that build on top of the vector register file, and on tighter integration with domain-specific accelerators for machine learning and cryptography.

6 Conclusion

This survey has examined the six most mature open-source implementations of the RISC-V Vector Extension that support 16-, 32-, and (where applicable) 64-bit elements. Ara/Ara2 stands as the reference high-performance design, Spatz as the leading compact cluster-oriented engine, Marian as the first open vector-cryptography solution, and Vicuna 2 as the most flexible embedded co-processor. RS5 illustrates a minimal-overhead integration path for microcontroller-class cores.

The availability of these designs under open licenses has dramatically lowered the barrier to experimenting with and deploying RVV. Continued community investment in verification, documentation, and multi-core integration will further accelerate the adoption of vector processing across the RISC-V ecosystem.

References

[1] RISC-V International, "The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture," Version of the Vector Extension, 2021–2024.

[2] M. Cavalcante et al., "Ara: A 1-GHz+ Scalable and Energy-Efficient RISC-V Vector Processor with Multiprecision Floating-Point Support in 22-nm FD-SOI," *IEEE Trans. VLSI Syst.*, vol. 28, no. 2, pp. 530–543, 2020.

[3] M. Perotti et al., "Ara2: Exploring Single- and Multi-Core Vector Processing with an Efficient RVV 1.0 Compliant Open-Source Processor," *IEEE Trans. Comput.*, 2024.

[4] M. Perotti et al., "Spatz: Clustering Compact RISC-V-Based Vector Units to Maximize Computing Efficiency," *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, 2025 (early access / arXiv:2309.10137).

[5] M. Platzer and P. Puschner, "Vicuna: A Timing-Predictable RISC-V Vector Coprocessor for Concurrent Real-Time Systems," and subsequent open-source release (vproc/vicuna).

[6] J. P. Jones et al., "Vicuna 2.0: A Configurable RISC-V Embedded Vector Hardware Platform," RISC-V Summit Europe, 2025.

[7] GAPH / PUCRS, "RS5: A Parameterizable 4-Stage Pipelined RISC-V Processor with Optional Zve32x Support," GitHub repository and associated theses, 2024–2025.

[8] T. Szymkowiak, E. Isufi, and M.-J. Saarinen, "Marian: An Open Source RISC-V Processor with Zvk Vector Cryptography Extensions," IACR ePrint 2024/1449; ACM CCS 2024 poster.